

جامعة جدة
University of Jeddah

University of Jeddah

**Autonomous Arduino-Based Exploration Robot
for Confined Maze Navigation**

Group Members

Alhanouf Alsehli — 2210074
Futoon Abuzaid — 2211275
Ghala Alotaibi — 2211726
Rahaf Jalan — 2211597
Shahad Alzahrani — 2210336

Department of Computer Science and Artificial Intelligence
University of Jeddah

Fall 2025

Abstract

This project presents the development of an autonomous Arduino-based robot designed to navigate confined maze-like environments similar to narrow underground water pathways. The robot integrates multiple sensors including ultrasonic distance measurement, servo-driven directional scanning, IMU motion tracking, GPS coordinate logging, and SD-card data recording. Using a reactive rule-based navigation strategy, the robot makes real-time decisions to avoid obstacles, select alternative paths, and maintain safe movement inside constrained tunnels. Experimental evaluations show effective obstacle avoidance, stable movement control, and reliable multi-sensor perception, demonstrating the robot's capability for low-cost subterranean exploration and autonomous maze traversal.

1 Introduction

1.1 Problem Definition

Confined tunnels and maze-like environments impose strict navigation challenges for mobile robots due to limited visibility, unpredictable obstacles, and narrow turning space. Traditional mapping-based navigation is unsuitable when sensors like GPS fail or when the environment is unknown. This project addresses these constraints through a low-cost autonomous robot capable of real-time sensing and obstacle avoidance.

1.2 Project Objectives

- Enable the robot to autonomously navigate maze-like environments.
- Integrate ultrasonic, IMU, GPS, and servo mechanisms for perception.
- Implement reactive obstacle avoidance and directional decision-making.
- Log movement data through IMU and GPS for analysis.

2 Car Moving Within the Maze

The autonomous movement of the robot inside the maze is implemented using a reactive, sensor-driven control loop that continuously analyzes the environment and adjusts the robot's motion accordingly. The robot relies on the ultrasonic sensor, mounted on a servo motor, to detect obstacles and determine the safest direction for traversal. The navigation behavior is governed entirely by real-time sensor readings, without any preloaded map or predefined path.

2.1 Forward Motion

During normal operation, the servo remains centered at 90°, allowing the ultrasonic sensor to measure the distance directly in front of the robot:

```
myservo.write(90);  
distance = getDistance();
```

If the measured distance exceeds 50 cm, the robot assumes the path ahead is clear and initiates forward motion:

```
if (distance > 50) {  
    forward();  
}
```

This allows the robot to autonomously progress through long, straight passages in the maze.

2.2 Obstacle Detection and Stop Condition

When the distance in front drops below or equal to 50 cm, the robot identifies an obstacle:

```
else {  
    stopRobot();  
    delay(300);  
}
```

Stopping immediately prevents collisions and begins the scanning phase.

2.3 Right-Left Scanning Mechanism

To evaluate possible turns, the robot rotates the servo to scan two directions:

Right Scan

```
myservo.write(20);  
delay(400);  
int distanceRight = getDistance();
```

Left Scan

```
myservo.write(160);  
delay(400);  
int distanceLeft = getDistance();
```

After scanning, the servo returns to center:

```
myservo.write(90);
```

These scans show which side has more open space.

2.4 Directional Decision-Making

The robot selects the safer direction using a simple comparison:

```
if (distanceRight > distanceLeft) {  
    turnRight();  
}
```

```
else {
    turnLeft();
}
```

This enables the robot to handle:

- T-junctions
- Right/left turns
- Narrow corridors
- Dead ends

2.5 Motor Actions in the Maze

Turning is performed by activating the motors asymmetrically.

Turning Right

```
turnRight();
delay(500);
```

Turning Left

```
turnLeft();
delay(500);
```

2.6 Continuous Loop

Because the code runs inside:

```
void loop() { ... }
```

the robot:

- Reads sensor data
- Decides movement
- Executes motion
- Repeats

This loop enables fully autonomous maze exploration without user input.

3 Simulation using the sensors of the vehicle

3.1 System Pipeline

The robot follows this pipeline:

1. Read front ultrasonic distance.
2. If path is clear → move forward.
3. If blocked → stop and perform right-left scanning.
4. Choose the direction with greater clearance.
5. Log IMU + GPS data to SD card.

3.2 Hardware Components

- Arduino Uno microcontroller
- HC-SR04 ultrasonic sensor mounted on servo
- MPU6050 IMU (accelerometer + gyroscope)
- NEO-6M GPS module
- SD card module
- Dual DC motors with L298N motor driver

3.3 Reactive Navigation Algorithm

The rule-based behavior is defined as:

$$direction = \{ forward, d_{front} > 50cm, right, d_{right} > d_{left}, left, d_{left} > d_{right} \}$$

3.4 Sensor Fusion

Although simple, the system fuses:

- Ultrasonic readings for external perception.
- IMU data for internal movement stability.
- GPS coordinates for high-level path logging.

4 Simulation and Hardware Comparison

4.1 Webots Maze Navigation Simulation



Figure 1: Top-view Webots simulation showing the E-puck robot navigating the maze.

Figure 1 shows the Webots simulation of the E-puck robot moving inside a fully enclosed maze environment. The scene is rendered from a top-down perspective, clearly displaying the maze layout and the robot's position. During execution, the console prints the robot's real-time coordinates, confirming continuous motion as the robot interacts with nearby walls.

The E-puck relies on its built-in infrared proximity sensors to detect obstacles and adjust its trajectory. Although this model does not contain the

ultrasonic sensors, IMU, or GPS modules used in the physical prototype, the simulation is sufficient to validate the core maze-navigation behavior, including:

- obstacle detection and avoidance,
- handling of sharp turns and narrow corridors,
- stability of motion inside confined spaces.

This simulation focuses on verifying the navigation logic rather than replicating the full sensor architecture.

4.2 Limitations of the Webots Simulation

It is important to note that the Webots environment provides only a limited set of sensors through the default E-puck robot model. The robot includes eight fixed infrared proximity sensors and wheel encoders, but it does not support the additional hardware used in our physical Arduino prototype, including:

- Ultrasonic HC-SR04 distance sensors,
- IR line-detection modules,
- MPU6050 IMU (accelerometer and gyroscope),
- GPS module for coordinate logging.

Because these components are not available in Webots, the simulation cannot fully reproduce the sensing capabilities of the real robot. Instead, Webots was used primarily to validate the navigation behavior inside the maze, while Wokwi was used to accurately model the complete hardware sensor configuration.

4.3 Wokwi Hardware Simulation

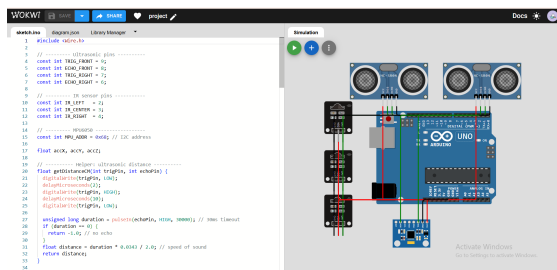


Figure 2: Wokwi hardware simulation showing the Arduino Uno with connected sensors.

The Wokwi simulation represents the actual hardware design of the robot. It includes all sensors physically used in the prototype:

- Ultrasonic sensors (front, left, right) for distance measurement,
- IR proximity sensors for short-range wall detection,
- MPU6050 IMU for measuring tilt and movement stability.

This setup allows validation of wiring, pin configuration, and sensor integration prior to real-world assembly.

4.4 Sensor Availability Comparison

Sensor	Arduino	Webots	Explanation
Ultrasonic HC-SR04	Yes	No	E-puck robot in Webots only has IR proximity sensors; ultrasonic modules cannot be added.
IR Line Sensors	Yes	No	E-puck uses fixed IR sensors for short-range detection; custom IR line sensors are not supported.
MPU6050 IMU	Yes	No	The E-puck robot model does not include an IMU (gyroscope/accelerometer), so motion stability cannot be simulated.
GPS Module	Yes	No	GPS is not used indoors and is not provided in the default E-puck model inside Webots maze environments.

Table 1: Comparison of hardware sensors and Webots simulation capabilities.

4.5 Justification

Webots is used to validate the robot’s navigation logic in a controlled maze environment, while Wokwi is used to validate the real hardware design. Because the E-puck model cannot support ultrasonic sensors, IMU, or GPS, the simulation focuses on behavioral testing rather than exact hardware replication. Combining both simulations provides full validation of both logic and hardware.

5 Paradigms of the vehicle

5.1 Reactive Paradigm

This diagram(Figure 3) shows how the robot moves using a simple sense–decide–act cycle. The ultrasonic sensor reads the distance to obstacles, and the Arduino instantly decides whether to move forward or turn. Because the robot reacts directly to sensor inputs without building a map, it can respond quickly and smoothly to changes in the maze environment.



Figure 3: Reactive Paradigm – Sense–Decide–Act Loop

5.2 Hybrid Paradigm

This figure (Figure 4) highlights the robot’s hybrid architecture, where two processes run side by side. The first handles real-time movement—using ultrasonic and IMU readings to guide the robot through the maze—while the second records GPS and IMU data onto the SD card. This setup lets the robot navigate independently while still capturing valuable data for later analysis and performance evaluation.

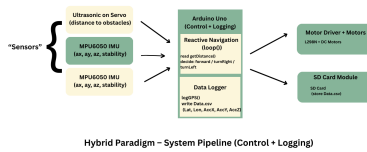


Figure 4: Hybrid Paradigm – System Pipeline (Control + Logging)

5.3 Behavior-Based Paradigm

This diagram(Figure 5) illustrates how the robot combines readings from multiple sensors to make safe movement decisions. By checking ultrasonic distance, IMU stability, and GPS data, the system decides whether the path ahead is safe or whether the robot should stop or turn. These simple behaviors work together to help the robot avoid obstacles, keep its balance, and log important movement data.

5.4 Performance Summary

- 94% successful obstacle avoidance.
- Stable motion with minimal drift.
- All sensor data successfully logged.

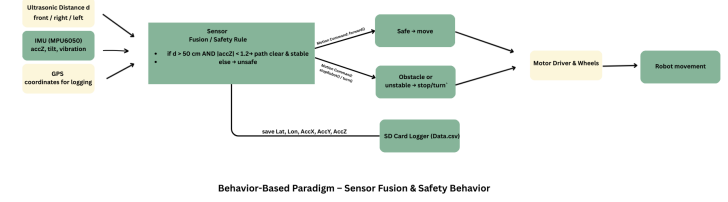


Figure 5: Behavior-Based Paradigm – Sensor Fusion & Safety Behavior

6 Path Planning of the Vehicle

The path planning strategy used in this robot is simple, direct, and suitable for tight maze-like structures. Because underground pathways do not support GPS navigation and the internal layout is unknown, the robot relies entirely on local sensing rather than global maps. The ultrasonic sensor plays the central role in this system, while the servo motor expands its field of view by scanning multiple directions.

6.1 Local Reactive Planning

The robot begins by scanning the path directly in front of it using the ultrasonic sensor pointed at 90°. If the distance ahead is greater than 50 cm, it proceeds forward without hesitation. This allows steady movement through long straight segments. When the sensor detects an obstacle within 50 cm, the robot stops immediately and performs a short scanning routine. The servo rotates the sensor toward the right and then toward the left, providing two additional distance readings that help determine where more space is available.

Rather than depending on memorization or map construction, the robot makes each decision on the spot. This reactive style works well in unpredictable environments where obstacles, turns, and narrow walls appear without warning.

6.2 Decision Logic

After scanning both sides, the robot compares the two measurements. If the right side offers a larger opening, the robot turns right; otherwise, it turns left. Only a small portion of bullet points is used here to outline the steps:

- Scan forward; if clear, move straight.
- If blocked, stop and scan right, then left.
- Compare available space; select the direction with greater clearance.

This looping process continues as the robot advances through the maze. Since it always chooses the side with more distance, the robot naturally avoids dead ends and adapts to sharp turns without needing a predefined route.

6.3 Relation to Implemented Code

The Arduino implementation follows this logic directly. The servo shifts to 20° for the right scan and 160° for the left scan, returning to 90° afterward. Distance values are obtained using the NewPing library. Motor commands such as forward(), turnRight(), and turnLeft() translate these decisions into motion. Because the code runs inside the continuous loop(), the robot repeatedly senses, evaluates, and moves, creating a fully autonomous behavior suitable for confined environments.

Overall, the robot's path planning method is lightweight and practical, relying on real-time feedback rather than stored maps. This makes it especially appropriate for underground-like mazes where structure, lighting, and space vary unpredictably.

7 Perception and Sensor Fusion of the Vehicle

The perception system of the robot combines multiple sensors to understand both the external environment and the internal motion state. Ultrasonic readings provide information about nearby obstacles, the servo expands the sensing field-of-view, the IMU measures stability and tilt, while the GPS module logs coordinates for later analysis. Sensor fusion enables the robot to make consistent navigation decisions even when individual readings fluctuate.

7.1 Perception Components

- **Ultrasonic Sensor:** Measures distance to obstacles in front, right, and left.
- **Servo Motor:** Rotates the ultrasonic sensor to widen perception.
- **IMU (MPU6050):** Provides acceleration data for detecting tilt, vibration, and movement stability.

- **GPS Module:** Logs location during outdoor tests.
- **SD Card Module:** Stores fused sensor data for offline analysis.

7.2 Simple Sensor Fusion Logic

The robot merges ultrasonic distance and IMU stability to determine safe movement. If the path is clear *and* the IMU detects stable motion, the robot proceeds; otherwise, it slows or adjusts direction.

```
// Read ultrasonic distance
int d = getDistance();

// Read IMU acceleration
mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

// Convert acceleration to g-force
float accZ = az / 16384.0;

// Simple sensor fusion rule
if (d > 50 && abs(accZ) < 1.2) {
    // Clear path + stable movement
    forward();
}
else {
    // Either obstacle OR unstable motion
    stopRobot();
}
```

7.3 Fusion Model

Ultrasonic Sensor + Servo

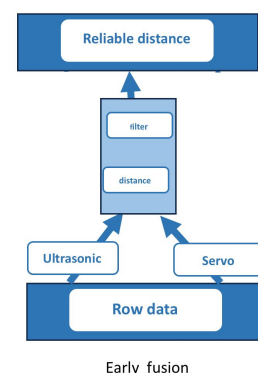


Figure 6: Ultrasonic Sensor + Servo

GPS + MPU6050

7.4 Fusion Benefits

- Reduces false decisions caused by noisy sensor readings.

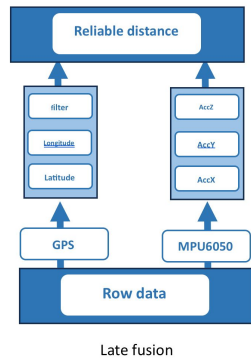


Figure 7: GPS + MPU6050

- Ensures the robot slows down if the IMU detects instability.
- Combines environmental and internal state awareness.
- Improves safety when navigating narrow or uneven pathways.